

Proyecto Ingeniería de Software – Sistema de gestión de Tareas

Andrés Felipe Florez Mora

C.C: 1092529261

Universidad de Pamplona

Facultad de Ingenierías y Arquitectura

Ingeniería del Software I - CR
Docente: Eduardo Rueda Fajardo
2026

1. Requerimientos de software

1.1 Documento SRS (Software Requirements Specification)

Propósito

Desarrollar una aplicación web para gestionar proyectos y tareas permitiendo la asignación de responsabilidades, seguimiento del progreso y colaboración entre usuarios.

Alcance

El sistema permitirá:

- Registro de usuarios.
- Inicio y cierre de sesión.
- Creación y visualización de proyectos.
- Creación, edición y eliminación de tareas.
- Asignación de tareas.
- Seguimiento del progreso.
- Notificaciones automáticas.
- Administración básica de usuarios.

Actores

Actor	Descripción
Administrador	Gestiona usuarios y proyectos
Usuario	Gestiona sus tareas y proyectos

REQUERIMIENTOS FUNCIONALES

	RF01
Nombre	Registro de usuarios
Propósito	Permitir que nuevos usuarios puedan crear una cuenta en el sistema.
Descripción	El sistema debe permitir que un usuario se registre proporcionando sus datos básicos para poder acceder a la plataforma.
Entrada	Nombre de usuario, correo electrónico y contraseña.
Salida	Usuario registrado correctamente en la base de datos.
Prioridad	Alta

Código del Requerimiento	RF02
Nombre	Inicio y cierre de sesión
Propósito	Permitir que los usuarios accedan y salgan del sistema de forma segura.
Descripción	El sistema debe validar las credenciales del usuario para permitir el acceso y también ofrecer la opción de cerrar sesión.
Entrada	Usuario y contraseña.
Salida	Acceso al sistema o mensaje de error si los datos son incorrectos.
Prioridad	Alta

Código del Requerimiento	RF03
Nombre	Crear proyectos o trabajos
Propósito	Permitir a los usuarios crear nuevos proyectos donde se organizarán las tareas.
Descripción	El sistema debe permitir que los usuarios creen proyectos indicando su nombre y descripción.
Entrada	Nombre del proyecto y descripción.
Salida	Proyecto creado y almacenado en el sistema.
Prioridad	Alta

Código del Requerimiento	RF04
Nombre	Visualizar lista de proyectos
Propósito	Permitir a los usuarios ver los proyectos existentes.
Descripción	El sistema debe mostrar una lista de los proyectos creados por el usuario o el equipo.
Entrada	Solicitud de visualización de proyectos.
Salida	Lista de proyectos disponibles.
Prioridad	Media

Código del Requerimiento	RF05
Nombre	Crear tareas
Propósito	Permitir agregar tareas dentro de un proyecto.
Descripción	El sistema debe permitir crear tareas especificando título, descripción, prioridad y fecha de entrega.
Entrada	Título, descripción, prioridad y fecha límite.
Salida	Tarea creada dentro del proyecto.
Prioridad	Alta

Código del Requerimiento	RF06
Nombre	Asignar tareas a usuarios
Propósito	Permitir distribuir responsabilidades entre los miembros del equipo.
Descripción	El sistema debe permitir asignar una tarea a un usuario específico del sistema.
Entrada	Selección de tarea y usuario asignado.
Salida	Tarea asignada correctamente.
Prioridad	Alta

Código del Requerimiento	RF07
Nombre	Actualizar estado de tareas
Propósito	Permitir modificar el progreso de una tarea.
Descripción	El sistema debe permitir cambiar el estado de una tarea (pendiente, en proceso o completada).
Entrada	Selección de tarea y nuevo estado.
Salida	Estado actualizado en el sistema.
Prioridad	Media

Código del Requerimiento	RF08
Nombre	Visualizar progreso de tareas

Propósito	Permitir a los usuarios conocer el avance de las tareas del proyecto.
Descripción	El sistema debe mostrar el progreso general de las tareas dentro de un proyecto.
Entrada	Solicitud de visualización del progreso.
Salida	Información del estado y avance de las tareas.
Prioridad	Media

Código del Requerimiento	RF09
Nombre	Editar tareas
Propósito	Permitir modificar la información de una tarea creada.
Descripción	El sistema debe permitir que los usuarios editen los datos de una tarea existente, como título, descripción, prioridad o fecha de entrega.
Entrada	Selección de tarea y nuevos datos a modificar.
Salida	Tarea actualizada correctamente en el sistema.
Prioridad	Media

Código del Requerimiento	RF10
Nombre	Eliminar tareas
Propósito	Permitir eliminar tareas que ya no sean necesarias dentro del proyecto.
Descripción	El sistema debe permitir a los usuarios eliminar una tarea existente dentro de un proyecto.
Entrada	Selección de la tarea a eliminar.
Salida	Tarea eliminada de la base de datos.
Prioridad	Media

Código del Requerimiento	RF11
Nombre	Recibir notificaciones de tareas asignadas
Propósito	Informar a los usuarios cuando se les asigne una nueva tarea o cuando una tarea tenga alguna actualización.
Descripción	El sistema debe enviar notificaciones a los usuarios cuando se les asigne una tarea o cuando se realicen cambios importantes en las tareas asignadas.
Entrada	Asignación o actualización de una tarea dentro del sistema.
Salida	Notificación mostrada al usuario en la plataforma.
Prioridad	Media

REQUERIMIENTOS NO FUNCIONALES

Código del Requisito	RNF01
Nombre	Usabilidad
Descripción	El sistema debe contar con una interfaz sencilla e intuitiva que facilite su uso por parte de cualquier usuario.
Prioridad	Alta

Código del Requisito	RNF02
Nombre	Rendimiento
Descripción	El sistema debe responder rápidamente a las acciones del usuario para garantizar una experiencia fluida.
Prioridad	Alta

Código del Requisito	RNF03
Nombre	Seguridad
Descripción	El sistema debe proteger los datos mediante autenticación y control de acceso de usuarios.
Prioridad	Alta

Código del Requisito	RNF05
Nombre	Disponibilidad
Descripción	El sistema debe estar disponible para los usuarios siempre que el servidor esté en funcionamiento.
Prioridad	Media
Código del Requisito	RNF04
Nombre	Compatibilidad
Descripción	El sistema debe funcionar correctamente en los principales navegadores web.
Prioridad	Media

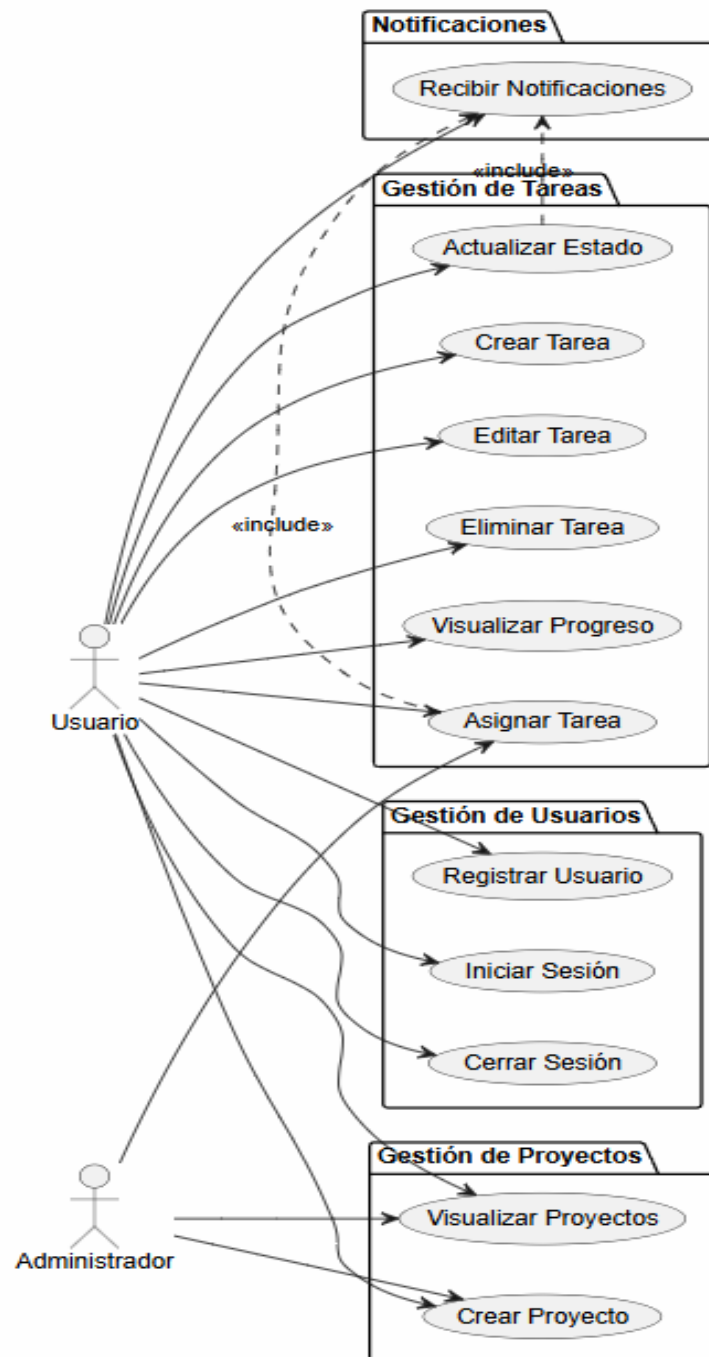
Código del Requisito	RNF06
Nombre	Escalabilidad
Descripción	El sistema debe permitir el aumento de usuarios y proyectos sin afectar el rendimiento.
Prioridad	Media

Código del Requisito	RNF07
Nombre	Integridad de datos
Descripción	El sistema debe garantizar la integridad de los datos almacenados, asegurando que la información de usuarios, proyectos y tareas se mantenga correcta, consistente y sin modificaciones no autorizadas dentro de la base de datos.
Prioridad	Alta

Matriz de trazabilidad

RF	Caso de Uso	Módulo	Prueba
RF01	CU01	Usuarios	CP01
RF02	CU02	Autenticación	CP02
RF03	CU03	Proyectos	CP03
RF04	CU04	Proyectos	CP04
RF05	CU05	Tareas	CP05
RF06	CU06	Tareas	CP06
RF07	CU07	Tareas	CP07
RF08	CU08	Reportes	CP08
RF09	CU09	Tareas	CP09
RF10	CU10	Tareas	CP10
RF11	CU11	Notificaciones	CP11

Modelo de casos de uso



2. Arquitectura de software

Arquitectura Seleccionada: Arquitectura Cliente-Servidor de Tres Capas

Capa de Presentación

- HTML5
- CSS3
- JavaScript
- Bootstrap

Capa de Lógica de Negocio

- Python
- Flask

Capa de Persistencia

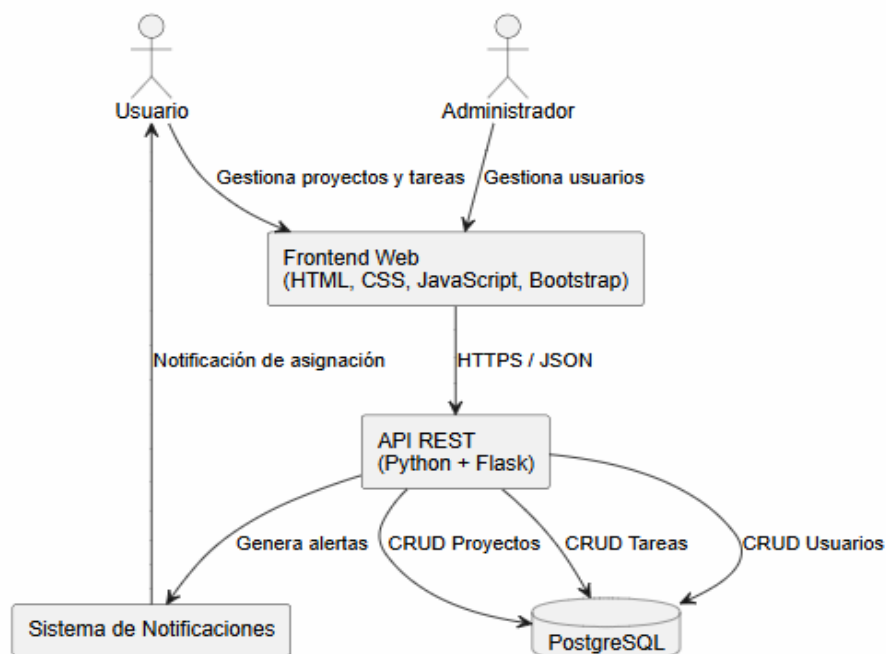
- PostgreSQL

Justificación

La arquitectura de tres capas permite:

- Separación de responsabilidades.
- Mantenimiento sencillo.
- Escalabilidad.
- Mayor seguridad.

Diagrama de Contenedores



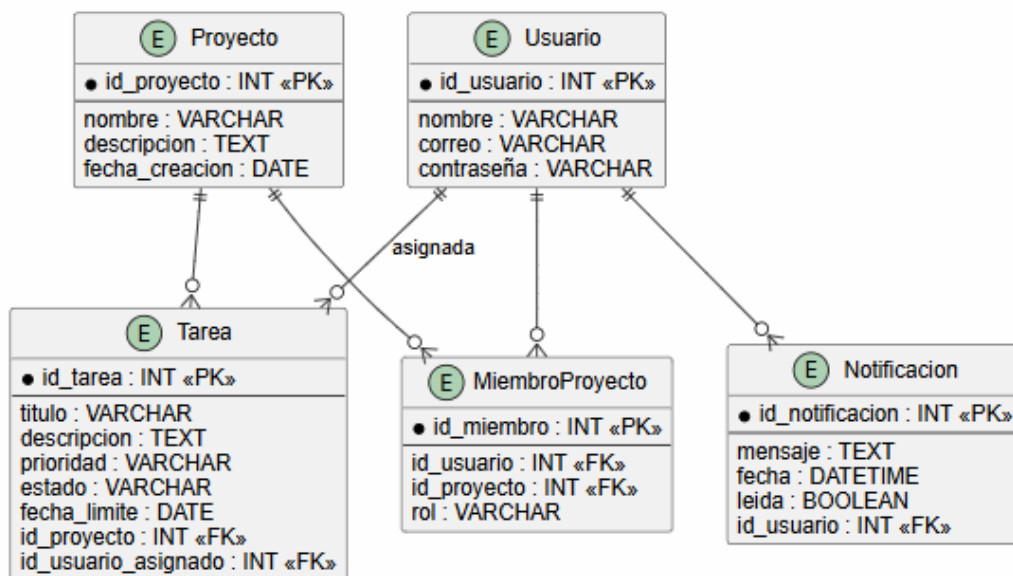
Registro de Decisiones (ADR)

Campo	Descripción
ID	ADR-001
Título	Selección del Framework Backend
Decisión	Utilizar Flask como framework para el desarrollo del backend del Sistema de Gestión de Tareas.
Justificación	Curva de aprendizaje baja. Framework ligero y flexible. Amplia documentación y comunidad de soporte. Fácil integración con PostgreSQL y APIs REST. Adecuado para proyectos académicos y de tamaño pequeño a mediano.
Consecuencias Positivas	Desarrollo más rápido. Menor complejidad arquitectónica. Facilidad de mantenimiento. Menor consumo de recursos.
Alternativas Evaluadas	Django, FastAPI, Node.js (Express).
Razón de Descarte	Django incorpora funcionalidades innecesarias para el alcance actual; FastAPI tiene una curva de aprendizaje mayor para el equipo; Express requeriría cambiar el lenguaje principal del backend.

Campo	Descripción
ID	ADR-002
Título	Selección del Sistema Gestor de Base de Datos
Decisión	Utilizar PostgreSQL como sistema gestor de base de datos relacional.
Justificación	Software gratuito y de código abierto. Alta seguridad y confiabilidad. Excelente rendimiento para aplicaciones web. Compatible con Flask mediante SQLAlchemy. Escalable para futuros crecimientos del sistema.
Consecuencias Positivas	Integridad y consistencia de los datos. Facilidad para realizar consultas complejas. Soporte para grandes volúmenes de información. Amplia documentación y soporte comunitario.
Alternativas Evaluadas	MySQL, SQLite, MongoDB.
Razón de Descarte	SQLite presenta limitaciones para múltiples usuarios concurrentes; MongoDB no se ajusta al modelo relacional requerido; PostgreSQL ofrece mejores características de integridad y escalabilidad.

3. Diseño de software

Diagrama de clases / Entidad-Relación



Contratos API

Método HTTP	Endpoint	Descripción	Entrada Principal	Respuesta Esperada
POST	/api/auth/register	Registrar un nuevo usuario en el sistema.	Nombre, correo, contraseña.	Usuario registrado correctamente.
POST	/api/auth/login	Autenticar un usuario existente.	Correo y contraseña.	Token de acceso o inicio de sesión exitoso.
POST	/api/proyectos	Crear un nuevo proyecto.	Nombre y descripción del proyecto.	Proyecto creado exitosamente.
GET	/api/proyectos	Obtener la lista de proyectos disponibles.	No requiere entrada.	Lista de proyectos registrados.
POST	/api/tareas	Crear una nueva tarea dentro de un proyecto.	Título, descripción, prioridad, fecha límite y proyecto asociado.	Tarea creada exitosamente.
PUT	/api/tareas/{id}	Actualizar la información de una tarea existente.	ID de la tarea y nuevos datos.	Tarea actualizada correctamente.
DELETE	/api/tareas/{id}	Eliminar una tarea del sistema.	ID de la tarea.	Confirmación de eliminación.
GET	/api/proyectos/{id}/progreso	Consultar el progreso de un proyecto.	ID del proyecto.	Porcentaje o estado de avance de las tareas.
GET	/api/notificaciones	Obtener las notificaciones del usuario.	Usuario autenticado.	Lista de notificaciones pendientes y leídas.

Prototipos UI/UX

Pantallas mínimas:

1. Login
2. Registro
3. Dashboard
4. Gestión de proyectos
5. Gestión de tareas
6. Detalle de tarea
7. Notificaciones
8. Perfil

4. Pruebas de software

Plan de pruebas

Objetivo: Verificar que los 11 requisitos funcionales operen correctamente.

Casos de prueba

ID	Caso	Resultado Esperado
CP01	Registrar usuario	Registro exitoso
CP02	Login válido	Acceso permitido
CP03	Crear proyecto	Proyecto creado
CP04	Crear tarea	Tarea creada
CP05	Asignar tarea	Tarea asignada
CP06	Actualizar estado	Estado actualizado

5. Mantenimiento

Problemas comunes:

- Error conexión BD.
- Error login.
- Servicio detenido.

Plan de Parches

Actualización mensual:

- Dependencias Python.
- PostgreSQL.
- Seguridad.

6. Gestión de ingeniería software

EDT / WBS

1. Análisis
2. Diseño
3. Desarrollo
4. Pruebas
5. Despliegue
6. Mantenimiento

Cronograma

Fase	Semanas
Planificación	1
Análisis	1
Diseño	2
Desarrollo	4
Pruebas	2
Despliegue	1

Matriz de riesgos

Riesgo	Probabilidad	Impacto
Retrasos	Media	Alta
Fallas BD	Baja	Alta
Cambios requisitos	Alta	Media

7. Modelos y métodos de Ingeniería del software

Campo	Descripción
ID	HU01
Nombre	Gestión de Proyectos
Como	Usuario
Quiero	Crear proyectos
Para	Organizar mis actividades y gestionar mejor mis tareas.
Prioridad	Alta
Estado	Aprobada
Criterios de Aceptación	1. El usuario puede crear un proyecto. 2. El usuario puede editar un proyecto existente. 3. El usuario puede eliminar un proyecto. 4. El sistema debe guardar los cambios realizados.

8. Proceso de ingeniería software

DoR

- Requisito definido.
- Prioridad asignada.
- Criterios de aceptación claros.

DoD

- Código desarrollado.
- Pruebas aprobadas.
- Documentación actualizada.

Retrospectiva

Lo que funcionó

- División de tareas.
- Uso de Git.

Mejoras

- Mejor estimación de tiempos.
- Más pruebas tempranas.

9. Calidad del software

Métricas

- Complejidad ciclomática < 10
- Cobertura > 80%
- Defectos críticos = 0

Análisis Estático

Herramientas:

- SonarQube
- Pylint

10. Seguridad de software

Amenaza	Mitigación
SQL Injection	Consultas parametrizadas
XSS	Sanitización de entradas
Robo de sesión	JWT
Fuerza bruta	Límite de intentos

SAST/DAST

Herramientas:

- Bandit
- OWASP ZAP

11. Economía de Ingeniería de software

Concepto	Costo
Desarrollo	\$0
Licencias	\$0
PostgreSQL	\$0
Flask	\$0
Hosting Académico	\$100.000

ROI

Beneficios

- Mejor organización.
- Reducción de retrasos.
- Mayor productividad.

ROI estimado

120% durante el primer año de uso.